

マルチプラットフォーム向け音声基盤移植

1. プロジェクト基本情報

項目	内容
プロジェクト概要	PlayStation 3向けゲームで使用されていた音声基盤を、FMODを利用したマルチプラットフォーム対応の音声基盤へ移植
対象プラットフォーム	Windows、PlayStation 5、Xbox Series、Nintendo Switch
開発言語	C++
ビルドシステム	CMake
使用技術	FMOD Studio API、FMOD Core API、各プラットフォームのオーディオAPI
担当領域	FMOD音声モジュールの設計・実装、既存音声APIとの接続、音声制御の意味変換、プラットフォーム別対応

2. 開発背景・目的・担当範囲

2.1 開発背景

オリジナル版のゲームはPlayStation 3専用のオーディオAPIと独自の音声ライブラリを使用しており、音声出力、ストリーミング、再生制御が同プラットフォームの仕組みに依存していました。そのため、ゲームを現行の複数プラットフォームへ移植するには、既存のゲームロジックから利用される音声機能を維持しながら、プラットフォームに依存する実装を置き換える必要がありました。

一方、ゲーム側がSE、BGM、カットシーンなどの制御に使用する上位の音声APIには、プラットフォーム固有の処理が含まれていませんでした。そのため、上位の音声制御とPlayStation 3固有の下位実装を分離できると判断しました。既存の音声APIと制御IDを維持したまま、自ら実装した中間層でゲーム側の要求をFMODの処理へ変換し、下位の音声ランタイムをFMODへ置き換える構成としました。

2.2 開発目的

- PlayStation 3固有の音声実装をFMODベースの実装へ置き換えます
- 既存のゲーム側音声APIと制御フローを維持し、呼び出し側の変更範囲を抑えます
- Windows、PlayStation 5、Xbox Series、Nintendo Switchで共通利用できる音声基盤を構築します
- 再生、停止、ポーズ、音量、Pitch、Filter、Reverb、時間軸など、既存ゲームが必要とする音声制御をFMOD上で再現します

2.3 個人の担当範囲

マルチプラットフォーム移植におけるFMOD音声モジュールについて、主に以下の設計・実装を担当しました。

- 既存音声システムの呼び出し関係と制御仕様の調査

- 既存音声APIからFMOD音声モジュールへ接続する互換レイヤーの実装

3. 技術調査と方式選定

3.1 既存音声基盤の調査

原版では、ゲーム側の音声制御層から独自音声ライブラリを経由し、PlayStation 3のオーディオポート、デコード、ストリーミング機能を利用していました。移植にあたり、再生や停止といった基本操作だけでなく、次の情報がゲームロジックと結び付いていることを確認しました。

- 音声制御IDによる再生インスタンスの管理
- Master、Dry、Wet、チャンネル別の音量
- Pitch、Filter、Envelope、Reverbの設定

この調査結果を基に、旧APIを単純にFMOD APIへ置換するのではなく、既存ゲームが期待する音声制御の意味をFMODのデータモデルへ対応付ける必要があると判断しました。

3.2 採用した移植方針

本プロジェクトでは、既存APIを維持しながら、その内部をFMODへ置き換える方針が採用されました。私はこの方針に基づき、互換レイヤー、FMOD共通処理、プラットフォーム固有処理を分離して実装しました。

この構成により、ゲームロジック側の変更範囲を限定しながら、Windows、PlayStation 5、Xbox Series、Nintendo Switchで共通の音声ランタイムを利用できるようにしました。一方、旧システムとFMODでは音声の管理単位とパラメーター表現が異なるため、既存の制御仕様を調査し、FMOD上で意味を再構成する必要がありました。

3.3 FMODを利用した構成

FMODには、マルチプラットフォーム向けの音声出力に加え、Event、Bank、Bus、DSP、Timelineなど、ゲーム音声の実装に必要な機能があります。これらを共通基盤として利用し、プラットフォームごとに音声再生機能を個別実装するのではなく、既存ゲーム固有の音声制御をFMODへ対応付けることに注力しました。

4. 全体の処理構成



FMOD音声モジュールの内部は、主に次の責務へ分割しました。

構成要素	主な責務
Module層	既存ゲームの音声IDとFMOD Event／Bankの対応付け、言語・プラットフォーム別のリソース選択
System層	FMODの初期化、毎フレーム更新、Bank poolとSound object poolの管理、プラットフォーム実装の切り替え
Sound object層	単一の音声制御IDに対応するEventInstance、再生状態、時間軸、音量、DSP設定の管理
Bank object層	Bankの登録、ファイルまたはパッケージ内メモリからのロード、アンロード、常駐属性の管理
Platform層	各プラットフォーム固有の初期化および設定

この分割により、既存ゲーム固有のID変換、FMOD共通処理、プラットフォーム固有処理をそれぞれ独立して扱えるようにしました。

5. 代表的な技術課題と解決方法

5.1 既存ゲーム側の変更範囲を抑えた音声ランタイムの置き換え

課題

既存の音声APIはゲーム内の多数の処理から利用されており、呼び出し側をFMOD APIへ直接変更すると影響範囲が大きくなります。一方で、旧実装はPlayStation 3固有の音声ライブラリとAPIに依存しているため、そのまま他

プラットフォームへ移植できませんでした。

判断

ゲーム側から見える音声API、音声制御ID、主要なパラメーターを維持し、互換レイヤーからFMOD音声モジュールへ処理を転送する構成としました。ただし、旧APIとFMOD APIを一対一で対応させるのではなく、既存ゲームが期待する動作単位で変換する方針としました。

実施内容

- 初期化と毎フレーム更新をFMOD Systemのライフサイクルへ接続しました
- 効果音とBGMの登録・再生を、音声IDからFMOD Event pathを解決する処理へ変換しました
- 再生、停止、強制停止、ポーズ、再開をEventInstanceの操作へ対応付けました
- 再生状態、再生時間、Eventの長さをFMODから取得し、既存形式へ返す処理を実装しました
- 既存APIの入力範囲を確認し、FMODへ渡す前に値を制限または変換しました

結果

ゲーム側の主要な音声制御フローを維持したまま、PlayStation 3固有の音声ランタイムをFMODベースの共通実装へ置き換えました。

5.2 音声IDとFMOD Event／Bankの対応付け

課題

既存ゲームでは、効果音、BGM、ボイスなどを複数の数値IDとチャンネル情報で識別していました。一方、FMODではBankとEvent pathを基に音声を管理するため、既存IDから適切なリソースを一意に解決する仕組みが必要でした。

実施内容

- 既存の音声IDを、Bank ID、Sound ID、Channel ID、音声種別を保持する内部表現へ変換しました
- 音声種別を判定し、効果音、BGM、ボイス、カットシーンなどに対応するEvent pathを生成しました
- 常駐Bankと必要時にロードするBankを区別し、不要になったBankをアンロードできるようにしました
- 通常のファイル読み込みに加え、パッケージ内のBankをメモリへ読み込み、FMODへ渡す経路を実装しました

結果

既存ゲームのID体系を維持しながら、FMODのBank／Event体系で音声リソースを管理できるようにしました。

5.3 既存の音声パラメーターをFMODへ変換

課題

旧システムとFMODでは、音量、Pitch、Filter、Envelope、Dry／Wet、Reverbなどのパラメーターの扱いが異なるため、既存APIから渡される値をFMODで扱える形式へ変換する必要がありました。

実施内容

- 既存音声APIから渡される音量とPitchをFMODで扱える形式へ変換し、音声種別やチャンネルに応じた音量制御と再生速度の制御を実装しました
- Filter、Envelope、Dry／Wet、Reverbの各設定をFMOD向けに変換し、再生時に適用しました

結果

既存ゲーム側の音声制御方法を維持しながら、音量、Pitch、Filter、Envelope、Dry／Wet、ReverbをFMOD上で扱えるようにしました。

6. 成果

- PlayStation 3固有の音声基盤を、FMODを利用したマルチプラットフォーム対応の音声基盤へ置き換えました
- 既存の音声APIと制御IDを維持する互換レイヤーを設け、ゲームロジック側の変更範囲を抑えました
- 効果音、BGM、ボイス、カットシーンのBank／Event管理と再生制御を実装しました
- 音量、Pitch、Filter、Envelope、Dry／Wet、Reverbなど、既存ゲームが使用する音声パラメーターをFMODへ対応付けました
- 共通処理とプラットフォーム固有処理を分離し、Windows、PlayStation 5、Xbox Series、Nintendo Switch向けの実装を同一モジュール内で管理できる構成としました

7. 本プロジェクトで得た知識・能力

7.1 FMODとゲーム音声システムへの理解

本プロジェクトで初めてFMODを用いたゲーム音声システムの開発に携わりました。Bankによる音声リソースの管理、Event／EventInstanceによる再生制御、毎フレームの更新、音量やPitchなどのパラメーター制御を実装し、ゲーム側の要求が音声ミドルウェア上で処理される一連の流れへの理解を深めました。